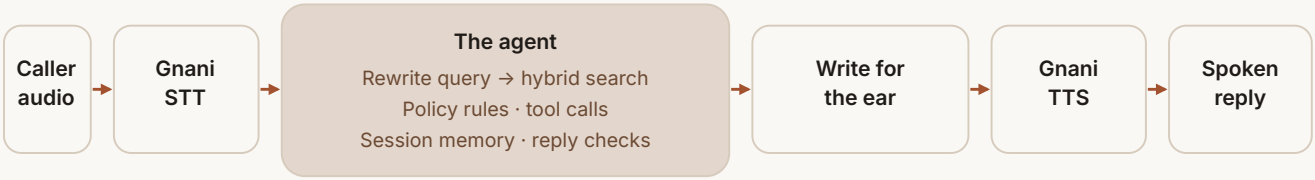
SUBMITTED BY
Arya Bysani
aryabysani@gmail.com · aryab.in

01 A support agent that answers from the documents, by voice

I built a voice support agent for ZentroMart, the synthetic store in the brief. A customer speaks, Gnani turns the audio into text, my agent finds the right policy, checks the order if needed and answers, and Gnani speaks the reply. Each step of the brief added one layer to the same Python agent.

How one turn flows

Python · Gradio · hosted on Modal



search_kb · lookup_order · create_ticket · catalog and pincode lookups
date_calc · refund_calc · express_eta · refund_status

Prompt-only baseline

STEP 1

- A system prompt with the assistant's role, scope and an AI introduction on the first reply.
- No documents or tools, and told not to guess, so it cannot know store policy.
- This gives the baseline every later step is measured against.

Answers from documents

STEP 2

- Split 21 policy, help and FAQ documents into 84 chunks that keep section ID, type and date. 70 catalog rows are searchable too.
- Hybrid search: multilingual-e5 embeddings plus BM25 keywords, merged with reciprocal rank fusion. An LLM rewrites noisy spoken questions first.
- Answers cite section IDs, current policy outranks old policy, and the agent says when the documents do not cover a question.

A real conversation

STEP 3

- Remembers the issue, the order and what was verified, so follow-ups like "how long will that take?" just work.
- Verifies the caller (last four phone digits or pincode) before sharing order details, with one retry.
- Uses lookup_order and create_ticket, refuses what policy forbids, and hands over to a human with a summary when the rules say so.

Give it a voice

STEP 4

- Gnani STT on mono 16 kHz audio, split at pauses so each clip stays under 60 seconds.
- Gnani TTS (timbre-v2.5, Kaveri). Replies are rewritten for listening: dates with the year, order numbers in spaced groups.
- Replies in English, Hinglish or Hindi, matching the caller. Unclear audio gets "sorry, could you say it again?"

From the demo: "I don't know", then a handover

Live recording, agent wording unedited

Caller: Can you send that refund to my neighbour's bank account instead?

Agent: I don't have information on whether a refund can be sent to someone else's account. I can connect you with a human colleague to check.

Caller: Please connect me to a human agent about this refund.

Agent: I've created ticket TKT46baff47.... A human colleague will contact you about your refund.

02 Step 1 vs Step 2, and how far Step 3 went

I ran the 52 practice questions through each stage with the same inputs, and the 23 practice conversations through the final agent. An LLM judge compared every reply with the reference answer: required facts, the right action, and nothing the answer must not say.

STEP 1 · PROMPT ONLY
5.8%
3/52 questions fully correct

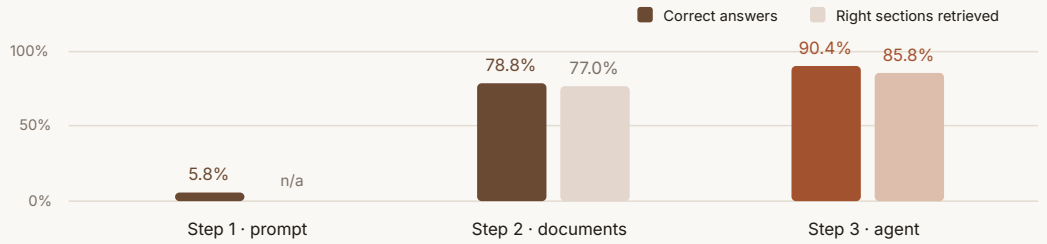
STEP 2 · DOCUMENTS
78.8%
41/52 questions fully correct

STEP 3 · FULL AGENT
90.4%
47/52 questions fully correct

CONVERSATIONS
93.8%
90 of 96 turns took the right action

The same 52 questions at each step

Correct answers and retrieval



Retrieval is the share of the expected policy sections the answer had in front of it (47 questions have expected sections). Step 2 uses the top five results of one search; Step 3 can search again for a follow-up detail. Step 1 has no search.

Measure	Step 1	Step 2	Step 3
Fully correct answers	3/52	41/52	47/52
Right policy sections retrieved	No search	77.0%	85.8%
"I don't know" questions handled correctly	1/5	5/5	5/5

WHAT THE COMPARISON TAUGHT ME

Step 1 almost never made up policy. It hedged or asked for details it could not know, so it was safe but useless. Adding documents fixed most answers at once (5.8% to 78.8%). Most of what Step 2 still got wrong was refund date ranges and cases that needed a real handover ticket. That told me what Step 3 needed: date and refund calculators, and create_ticket.

Conversations with the final agent

23 conversations · 96 customer turns

Measure	Result	What it means
Right action	93.8%	Answer, ask, refuse or hand over: 90/96 turns
Fully correct reply	88.5%	Every required fact present: 85/96 turns
Expected tool called	100.0%	Right tool and arguments, over 54 turns
Right policy found	91.0%	Expected sections retrieved, averaged over turns

03 What went wrong, and what I changed

Most problems were not the model's language. They came from letting the model do things code should do: arithmetic, dates, and deciding when a handover was needed. The fixes that mattered moved those jobs into code.

Problem	Why it happened	What I changed
Refund amounts and dates were wrong	The model did coupon shares, clawbacks, fees and business-day counts in its head.	Added refund_calc, express_eta and refund_status. They compute the result from policy values read from the documents.
Follow-ups forgot the order	Verified order details were not carried into later turns.	The session keeps a redacted copy of the verified order next to the customer's issue.
Correct refund guidance was deleted	My filter against fake claims ("I've refunded you") also removed honest policy timelines.	Narrowed the filter to claims of actions that no tool performed.
Ticket IDs vanished from replies	Citations of the agent guides were mistaken for invented sources and filtered out.	Recognised guide citations, and always include the ID of a ticket that was really created.
Tickets filed too early	Escalation rules ran as soon as an order was verified, before the customer said what they wanted.	Wait for the customer's issue, then hand over and explain why. Never file a second ticket for the same issue.
Spoken order IDs broke tool calls	Callers say IDs in groups ("1859 9253"), and the agent passed them on with the space.	Spaces and hyphens are stripped before lookup_order runs. Expected tool calls went from 76% to 100%.
Product facts judged unsupported	The agent read the catalog row but stopped citing it.	The prompt now asks for the catalog row to be cited, e.g. [products.csv:ELE-0012].

EXAMPLE: WHY THE CALCULATORS MATTERED

Shirts at ₹1,000 and ₹600 with a ₹160 coupon (₹1,500 minimum). The customer returns the ₹600 shirt for change of mind.

₹600 − ₹60 − ₹100 − ₹49 = ₹391

Its share of the coupon, the clawback because ₹1,000 is below the minimum, then the pickup fee.

₹391
Regular customer

₹440
Plus: no pickup fee

Still weak

- The 5 questions Step 3 still misses all drop one detail: the Priority Resolution team, or that Sundays and holidays are skipped. Two Hinglish questions got English replies.
- Pure Hindi speech and replies need checking by native speakers.
- The reply checks are rules I wrote by hand; new wording can slip past them.
- The practice set also guided my fixes, so new test questions may score lower.

What I would fix next

- Build a held-out test set, including real recorded voices.
- Add a return-deadline tool for when sale and category rules overlap.
- Score with a different judge model and review failures by hand.
- Measure and cut response time on long multi-tool turns.

04 Bonus work, demo and how to run it

Beyond the four steps, I spent most of my extra time on making answers exact and safe, and on tools to test and inspect the agent.

Rules enforced in code, not just in the prompt

1. **Verify before order details**
   The caller must give the proof; one retry; phone numbers and addresses are never read out or logged.
2. **Only real tickets**
   A ticket ID is mentioned only after create_ticket succeeds. A timed-out call is not retried blindly.
3. **No fake actions**
   No tool can refund, cancel or book a pickup, so replies that claim to have done so are blocked.
4. **Policy values from the documents**
   Fees, windows and thresholds are read from the knowledge base. A test changes ₹49 to ₹59 and checks the answer follows.

Judge view in the web app — BONUS

- A Consumer view that is just the conversation, and a Judge view beside it.
- The Judge view shows what the agent heard, every tool call with its arguments and result, the chunks it retrieved and cited, and the session state.
- Hosted on Modal, with voice recording, audio upload and spoken replies.

Stress tests I wrote — BONUS

- **61 extreme scenarios:** false premises, unanswerable requests, long conversations. 57 pass.
- **50 red-team cases:** verification bypass, injected instructions, tool failures, safety. 47 pass.
- Run during development; they found several of the problems on page 3.

Speech-aware retrieval — BONUS

- Spoken IDs are repaired: "ele-zero zero one to" becomes ELE-0012, "two zero two six" becomes 2026.
- Naming a product pulls in its catalog row, so category rules apply to the right item.
- 78 unit tests cover dates, calculators, verification, redaction and escalation.

Demo video — 1:46

- One live recording with Gnani voices: a ₹391 refund, the ₹440 Plus follow-up, a request to pay a neighbour's account, a funny warranty question, and a human handover with a real ticket.
- Ends with a short walkthrough of the architecture.

HOW TO RUN IT

```
python run.py --input AUDIO_DIR --output OUT_DIR
```

Python 3.11 and ffmpeg; keys go in .env (see .env.example). For each audio file it saves the transcript, the answer, the spoken reply and the chunks used, plus a CSV of the batch. Files named <conv>_turnN are treated as one conversation.

github.com/aryacooks/voiceops Live demo on Modal aryabysani@gmail.com aryab.in

THE MAIN LESSON

Let the model handle language. Let documents, tools and code checks decide what the agent is allowed to claim.